Основы программирования на C++

Занятие 5. Динамическое выделение памяти

Дерево языка

В предыдущей лекции…

Указатель - это адрес переменной в памяти.

int* → храним адрес ‘int’

Операции

- &x — Взятие адреса
- *x — Разыменование

В предыдущей лекции…

```c
#include <stdio.h>
int main()
{
   int x = 10;
   int* y = &x;
   printf("x = %d\n", x);
   *y = *y + 1;
   printf("x = %d\n", x);
   return 0;
}
```

Результат

```
x = 10
x = 11
```

В предыдущей лекции…

```c
#include <stdio.h>

void swap(int *a, int *b)
{
    int tmp = *b;
    *b = *a;
    *a = tmp;
}


int main()
{
    int a = 10, b = 15;
    printf("a = %d, b = %d\n", a, b);
    swap(&a, &b);
    printf("a = %d, b = %d\n", a, b);
    return 0;
}
```

Результат

```
a = 10, b = 15
a = 15, b = 10
```

Предисловие

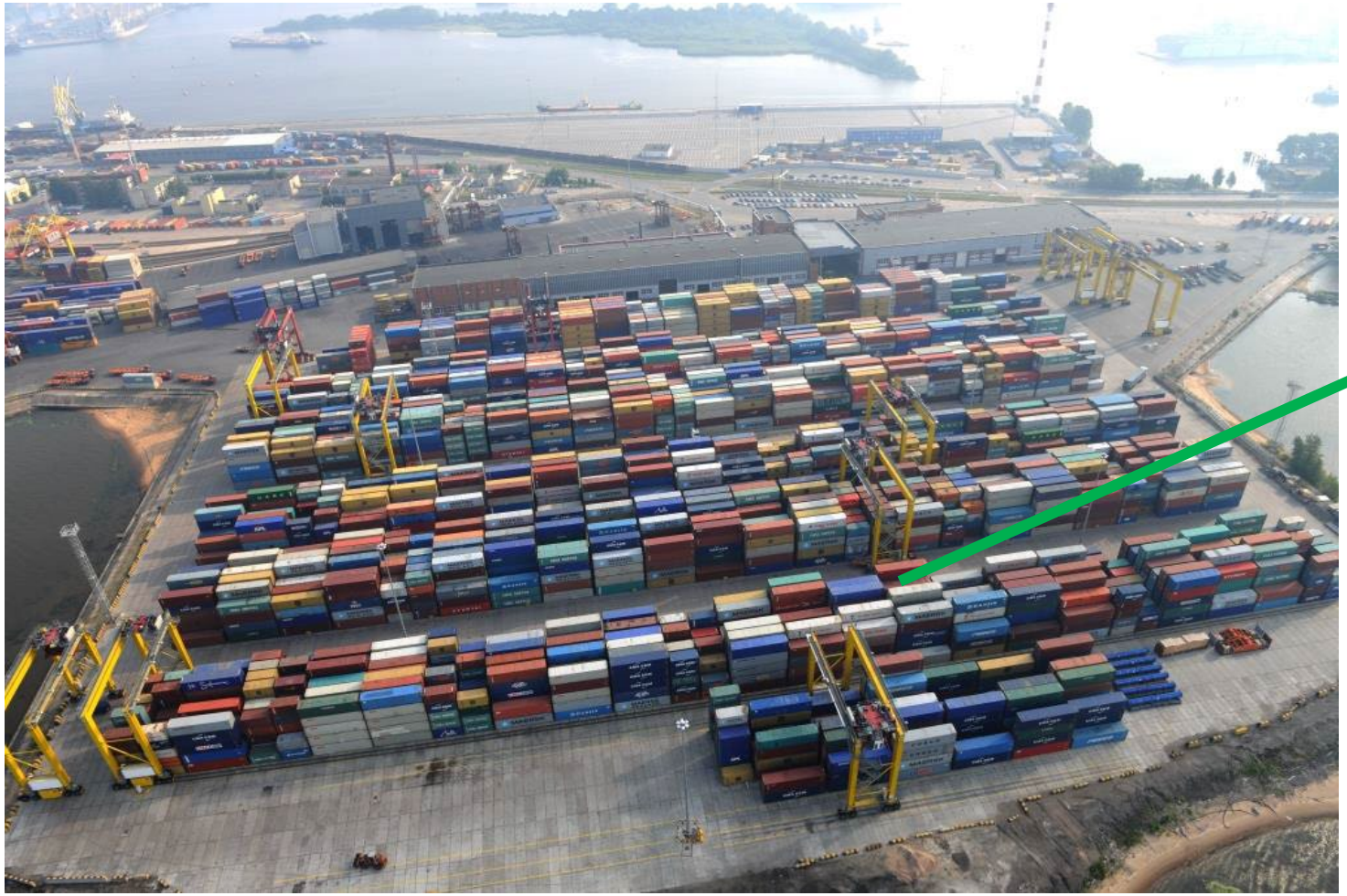
```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

const int N = 10;

void input_array(double *x)
{
     int arrMax = 100, arrMin = 0;
     for (int i = 0; i < N; i++)
     x[i] = arrMin + (arrMax - arrMin) * ((double) rand() / RAND_MAX);
}

void print_array(double* x)
{
     for (int i = 0; i < N; i++)
          printf("%lf ",x[i]);
     printf("\n ");
}

int main()
{
     srand(time(NULL));
     double array[N];
     input_array(array);
     print_array(array);
     return 0;
}
```

Существует ли ограничение на значение N?

Устройство памяти процесса

Процесс – программа во время исполнения и выделенные ей ресурсы

Устройство памяти процесса



Монитор ресурсов

Процессы — Использование физической памяти: 84%

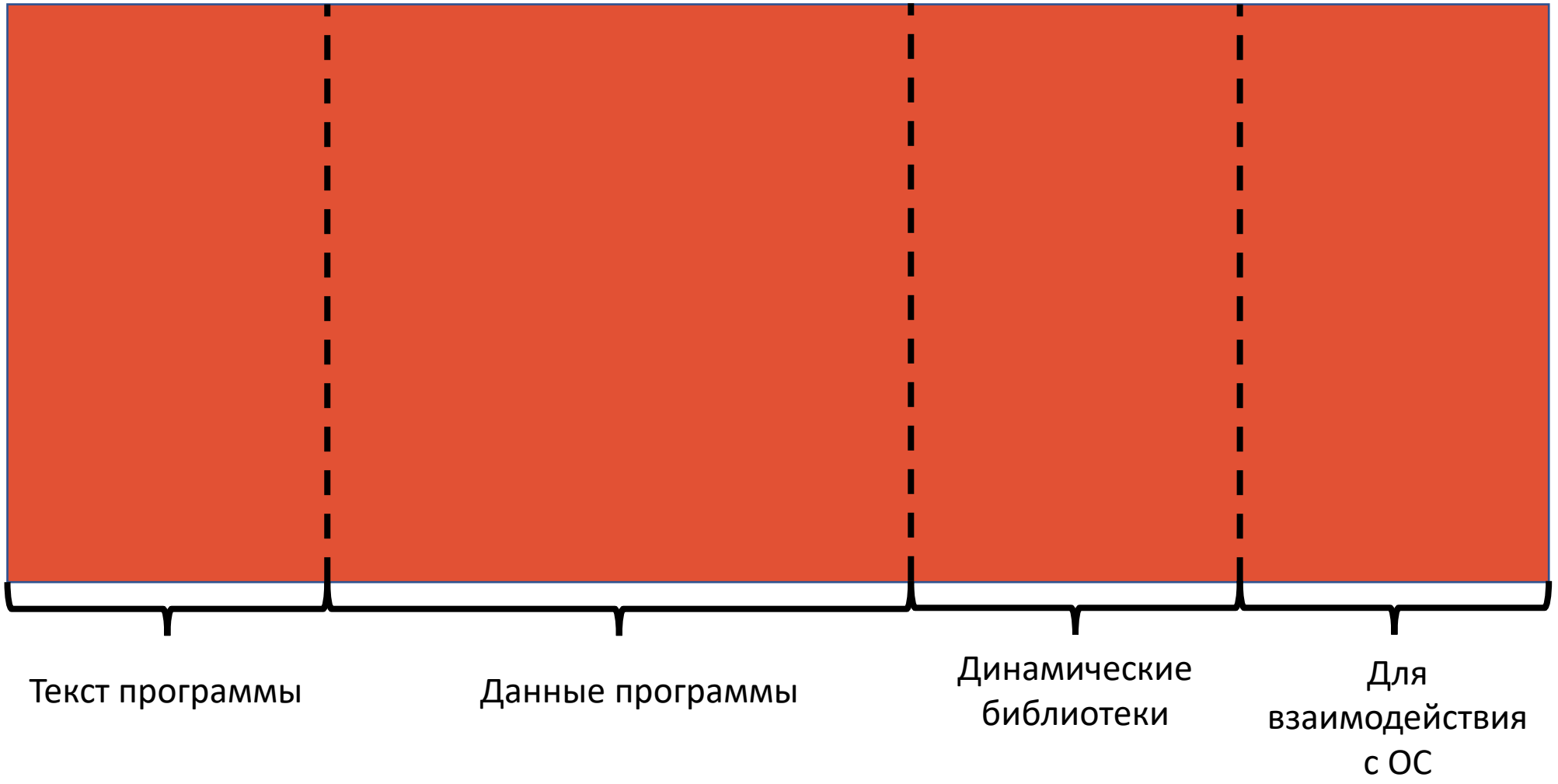
Образ	ИД пр...	Ошибок страницы физической памяти/сек	Завершено (КБ)	Рабочий набор (...	Общий (КБ)	Частный (КБ)
devenv.exe	11508	0	537 172	628 336	222 556	405 780
vmmem	4760	0	428 960	329 152	812	328 340
firefox.exe	16700	0	419 976	358 532	78 952	279 580
MsMpEng.exe	6056	0	1 356 920	309 772	80 600	229 172
firefox.exe	6300	0	237 656	285 944	115 724	170 220
POWERPNT.EXE	19216	0	210 532	284 912	122 380	162 532
firefox.exe	2296	0	157 132	188 492	61 884	126 608
firefox.exe	1500	0	111 740	150 912	59 040	91 872
dwm.exe	9832	0	129 664	140 364	52 556	87 808
firefox.exe	14496	0	109 544	143 216	58 896	84 320
Memory Compression	3436	0	904	57 128	0	57 128
firefox.exe	15608	0	90 704	106 468	58 644	47 824
perfmon.exe	10120	0	43 560	60 804	18 948	41 856
explorer.exe	2012	0	133 304	153 720	115 416	38 304
ServiceHub.SettingsHost.exe	13764	0	75 068	108 408	72 436	35 972
ServiceHub.IdentityHost.exe	15856	0	60 644	85 324	51 880	33 444
WindowsTerminal.exe	10264	0	61 228	95 028	61 740	33 288
ScreenSketch.exe	4492	0	47 092	87 428	55 056	32 372
ServiceHub.ThreadedWaitDialo...	12656	0	76 304	100 632	68 492	32 140

Физическая память — Используется: 6707 МБ — Доступно: 1191 МБ

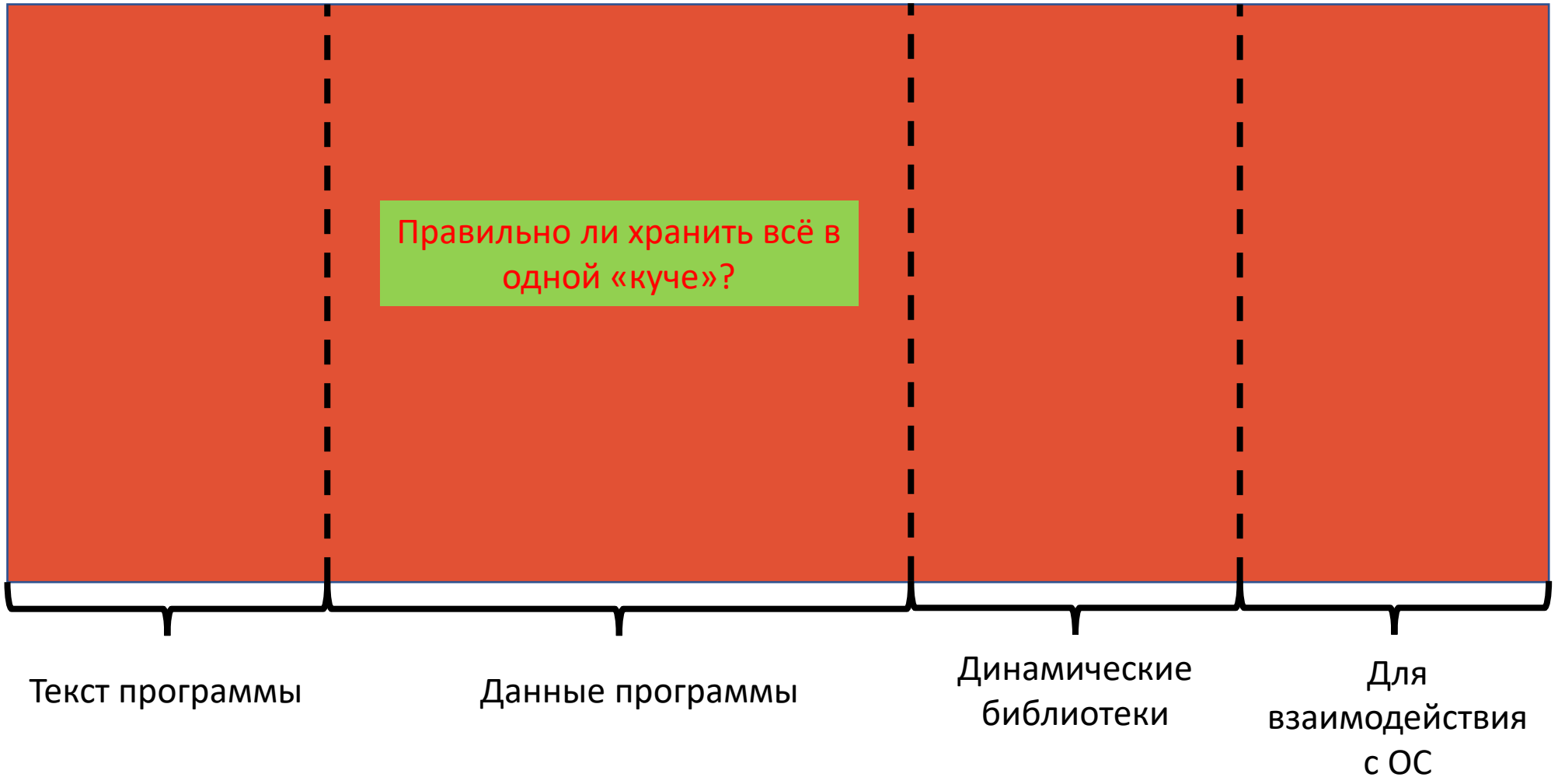
Зарезервировано аппаратно	Используется	Изменено	Ожидание	Свободно
262 мегабайт	6707 мегабайт	32 мегабайт	1185 мегабайт	6 мегабайт

Доступно	1191 мегабайт
Кэшировано	1217 мегабайт
Всего	7930 мегабайт
Установлено	8192 мегабайт

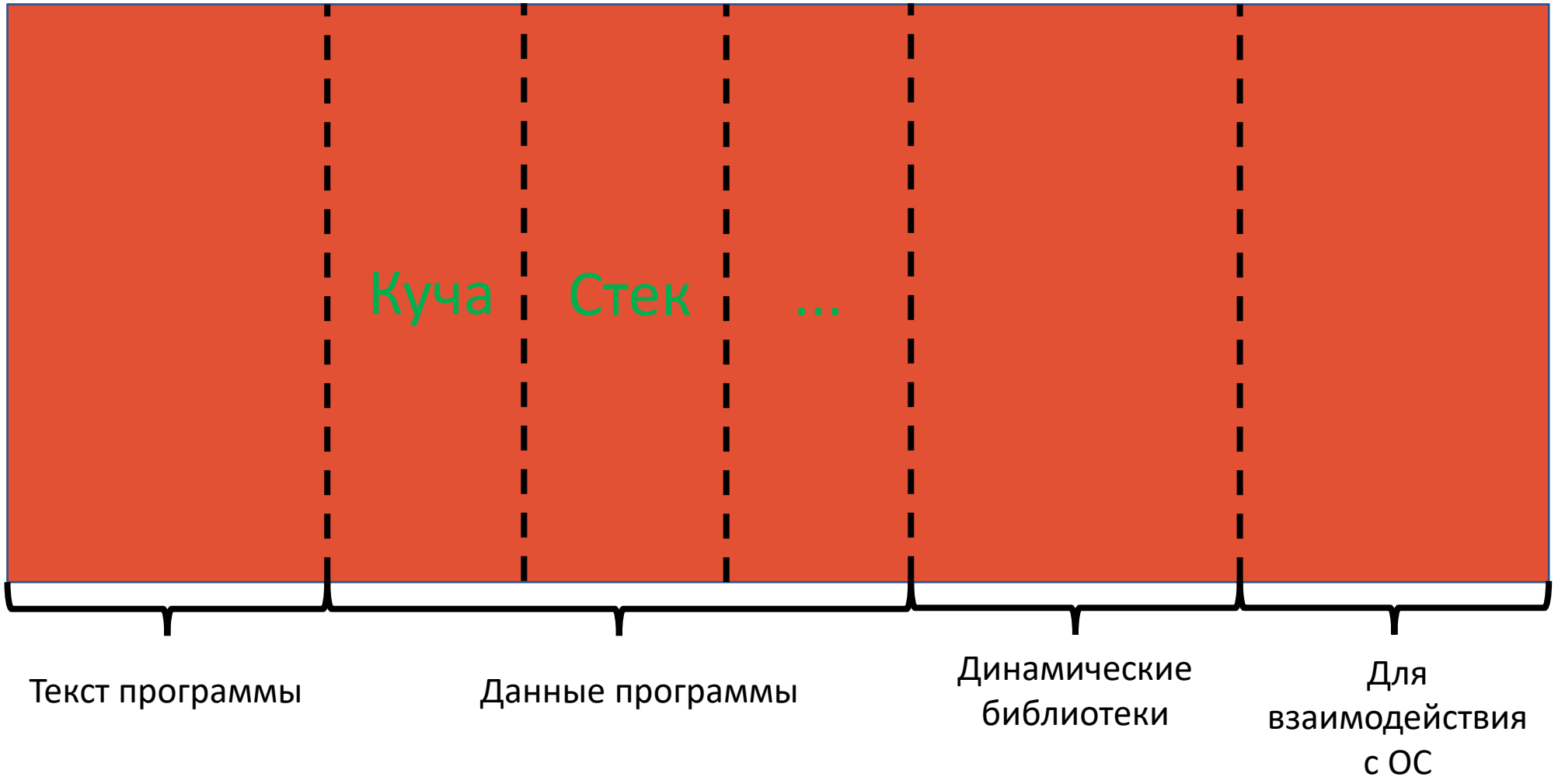
Устройство памяти процесса



Устройство памяти процесса



Устройство памяти процесса

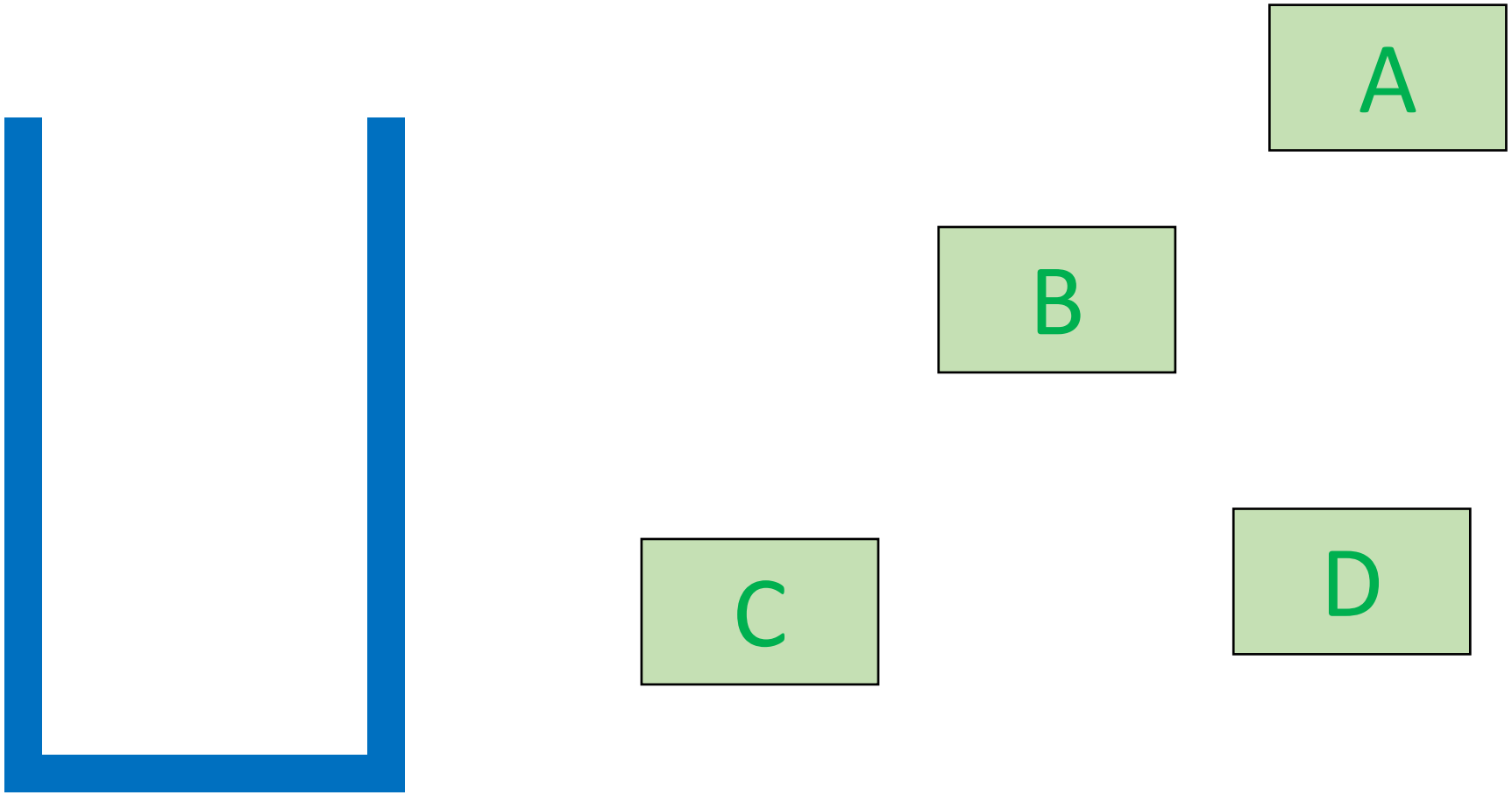


```
55e233729000-55e23372a000 r--p 00000000 00:2e 10696049115018075          /mnt/c/Users/79629/Documents/ubuntu_files/memory
55e23372a000-55e23372b000 r-xp 00001000 00:2e 10696049115018075          /mnt/c/Users/79629/Documents/ubuntu_files/memory
55e23372b000-55e23372c000 r--p 00002000 00:2e 10696049115018075          /mnt/c/Users/79629/Documents/ubuntu_files/memory
55e23372c000-55e23372d000 r--p 00002000 00:2e 10696049115018075          /mnt/c/Users/79629/Documents/ubuntu_files/memory
55e23372d000-55e23372e000 rw-p 00003000 00:2e 10696049115018075          /mnt/c/Users/79629/Documents/ubuntu_files/memory
55e2350fc000-55e23511d000 rw-p 00000000 00:00 0                          [heap]
7f16bfce1000-7f16bfd06000 r--p 00000000 08:10 11116                      /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f16bfd06000-7f16bfe7e000 r-xp 00025000 08:10 11116                      /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f16bfe7e000-7f16bfec8000 r--p 0019d000 08:10 11116                      /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f16bfec8000-7f16bfec9000 ---p 001e7000 08:10 11116                      /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f16bfec9000-7f16bfecc000 r--p 001e7000 08:10 11116                      /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f16bfecc000-7f16bfecf000 rw-p 001ea000 08:10 11116                      /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f16bfecf000-7f16bfed5000 rw-p 00000000 00:00 0
7f16bfedd000-7f16bfede000 r--p 00000000 08:10 11000                      /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f16bfede000-7f16bff01000 r-xp 00001000 08:10 11000                      /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f16bff01000-7f16bff09000 r--p 00024000 08:10 11000                      /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f16bff0a000-7f16bff0b000 r--p 0002c000 08:10 11000                      /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f16bff0b000-7f16bff0c000 rw-p 0002d000 08:10 11000                      /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f16bff0c000-7f16bff0d000 rw-p 00000000 00:00 0
7ffc901b0000-7ffc901d1000 rw-p 00000000 00:00 0                          [stack]
7ffc901e6000-7ffc901ea000 r--p 00000000 00:00 0                          [vvar]
7ffc901ea000-7ffc901eb000 r-xp 00000000 00:00 0                          [vdso]
```

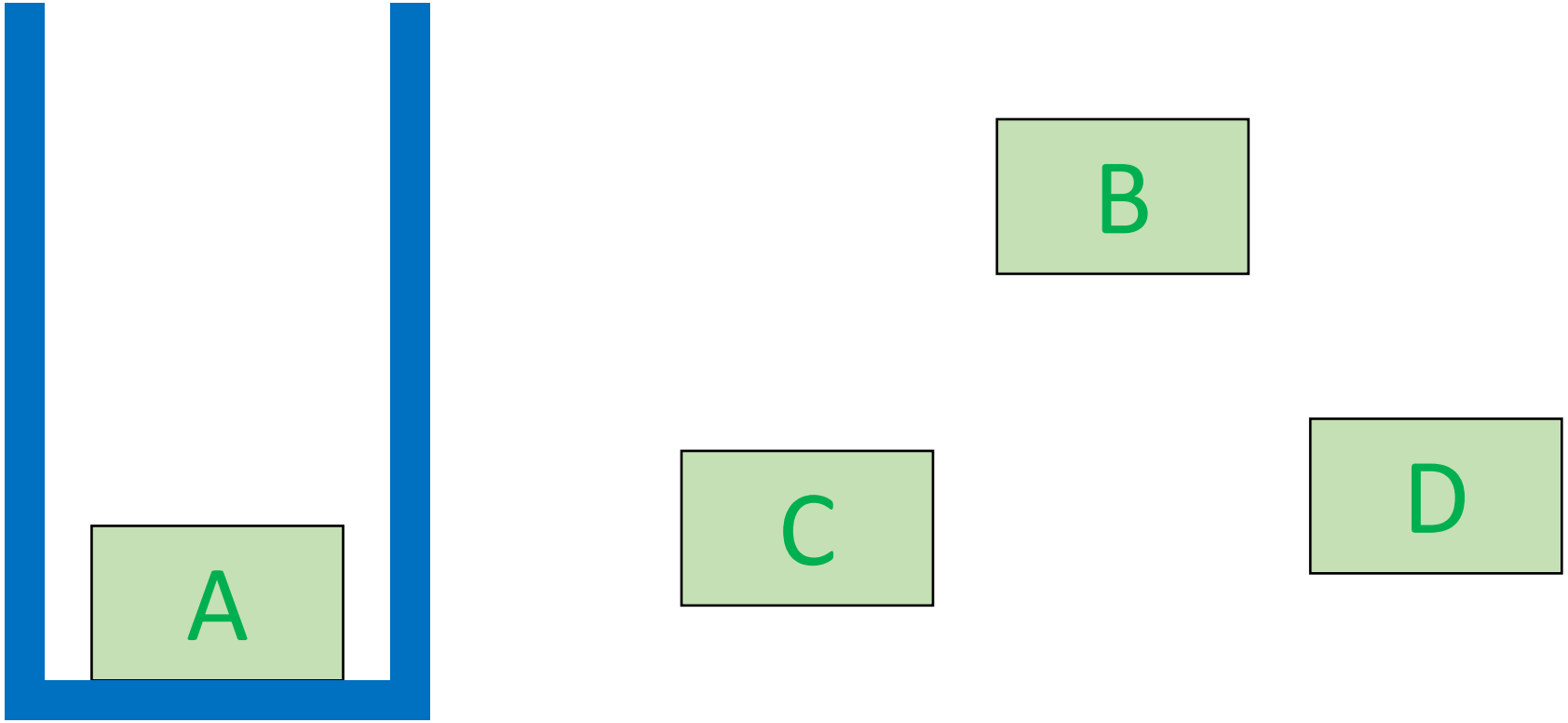
Стек

Стек — это структура данных, которая работает по принципу FILO (first in — last out; первый пришел — последний ушел).

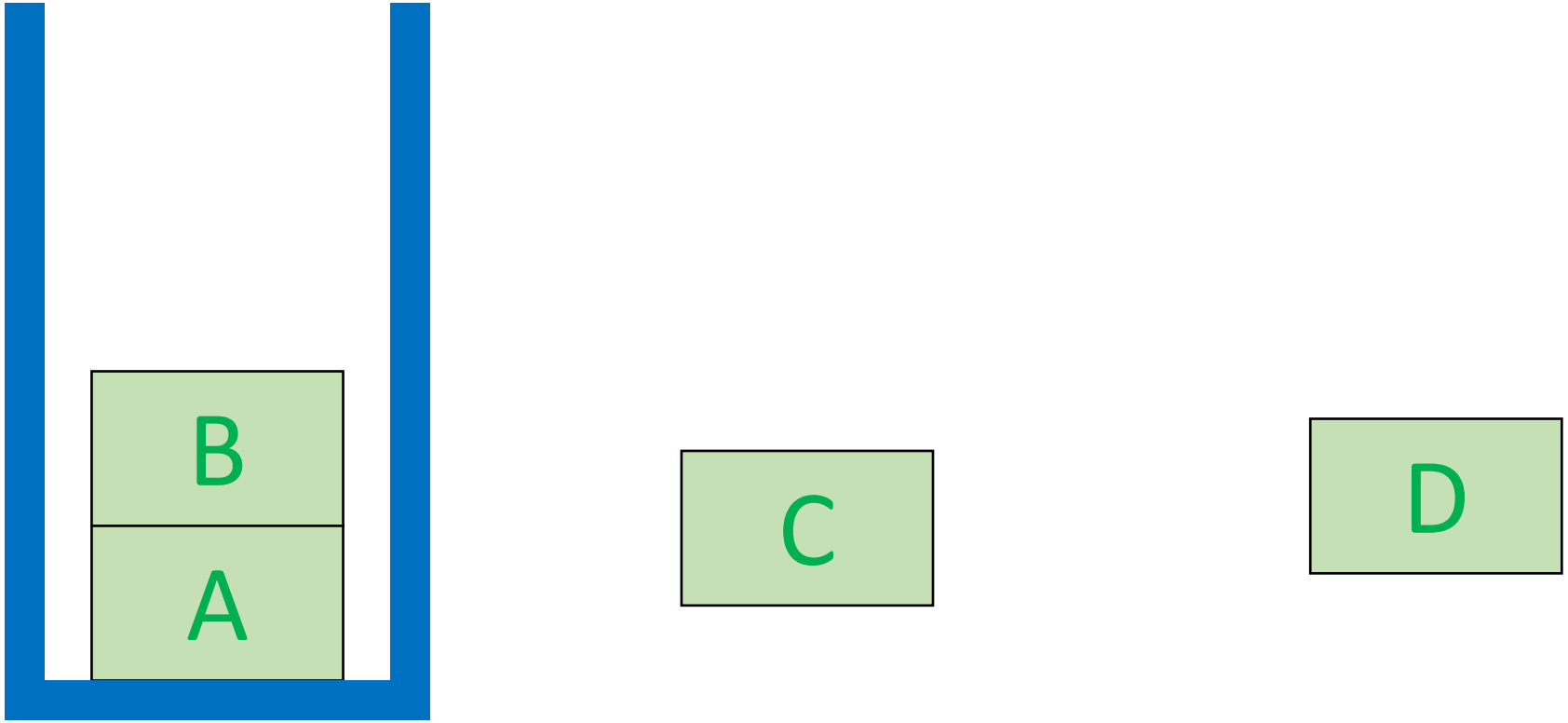
Стек



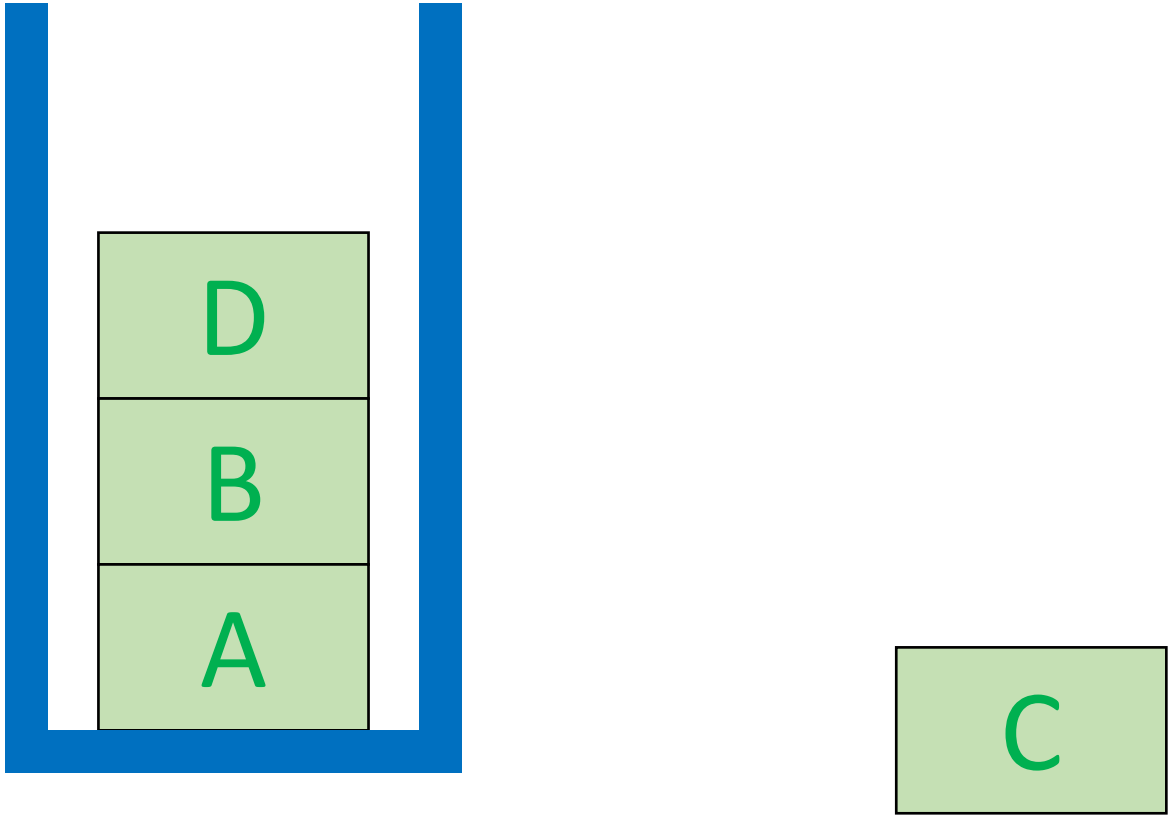
Стек



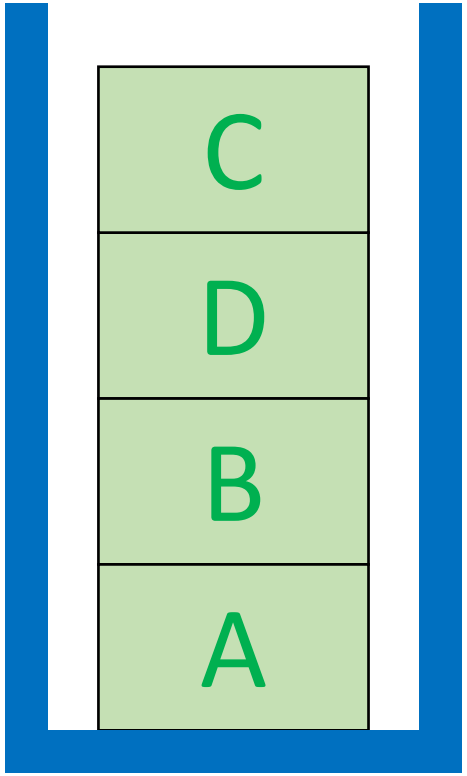
Стек



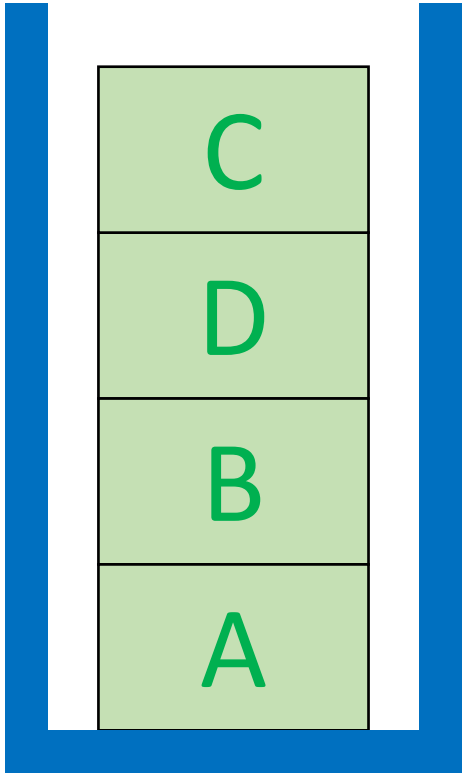
Стек



Стек

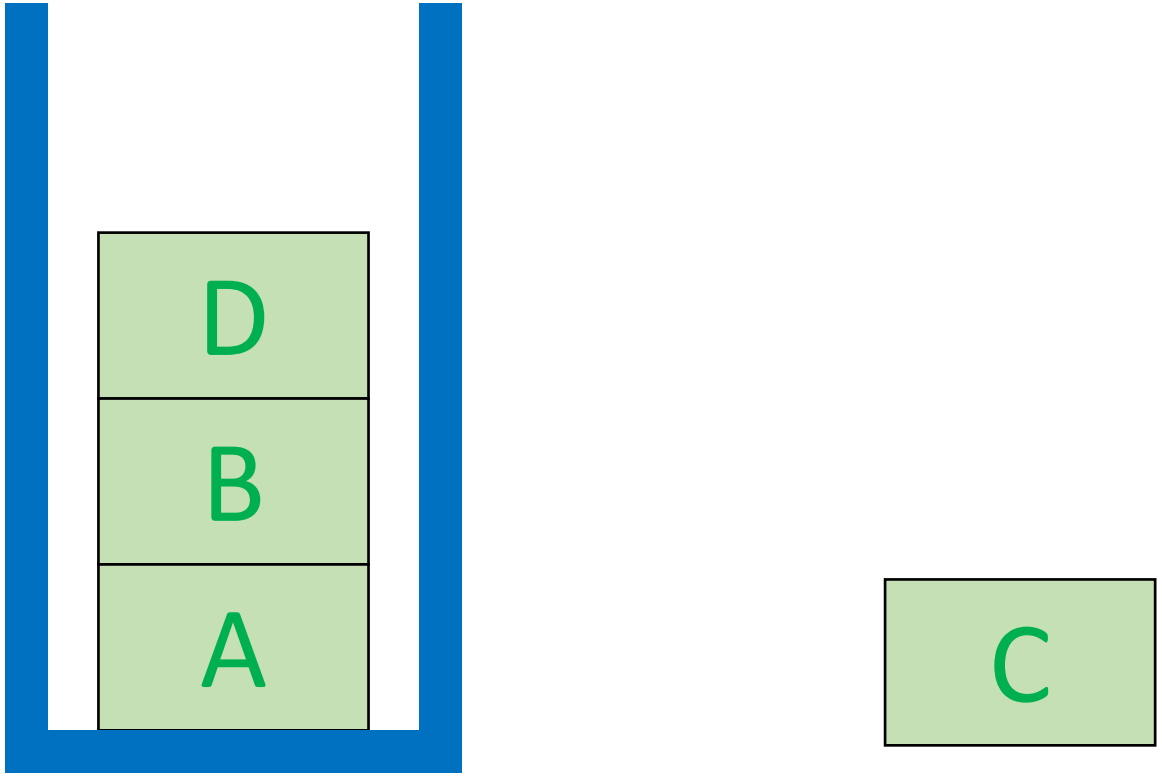


Стек

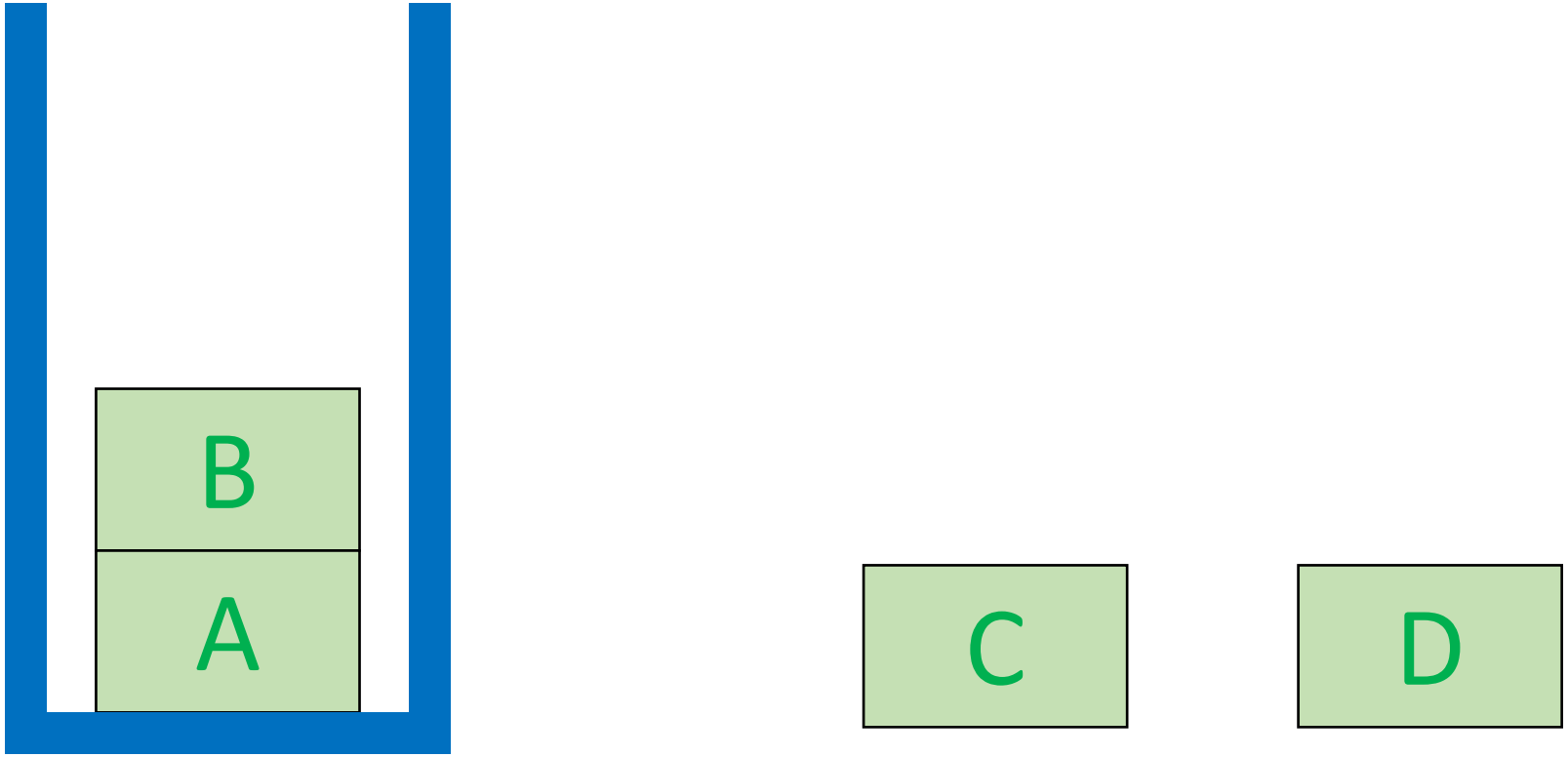


Как достать «B»?

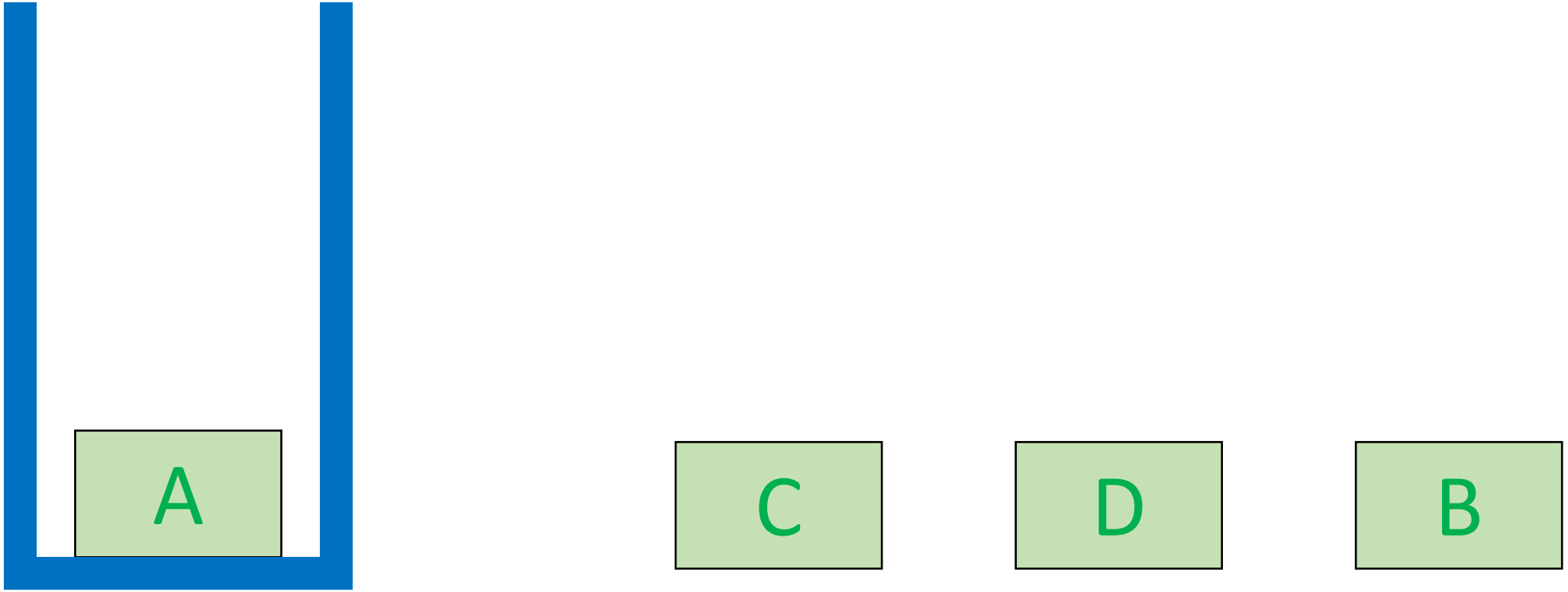
Стек



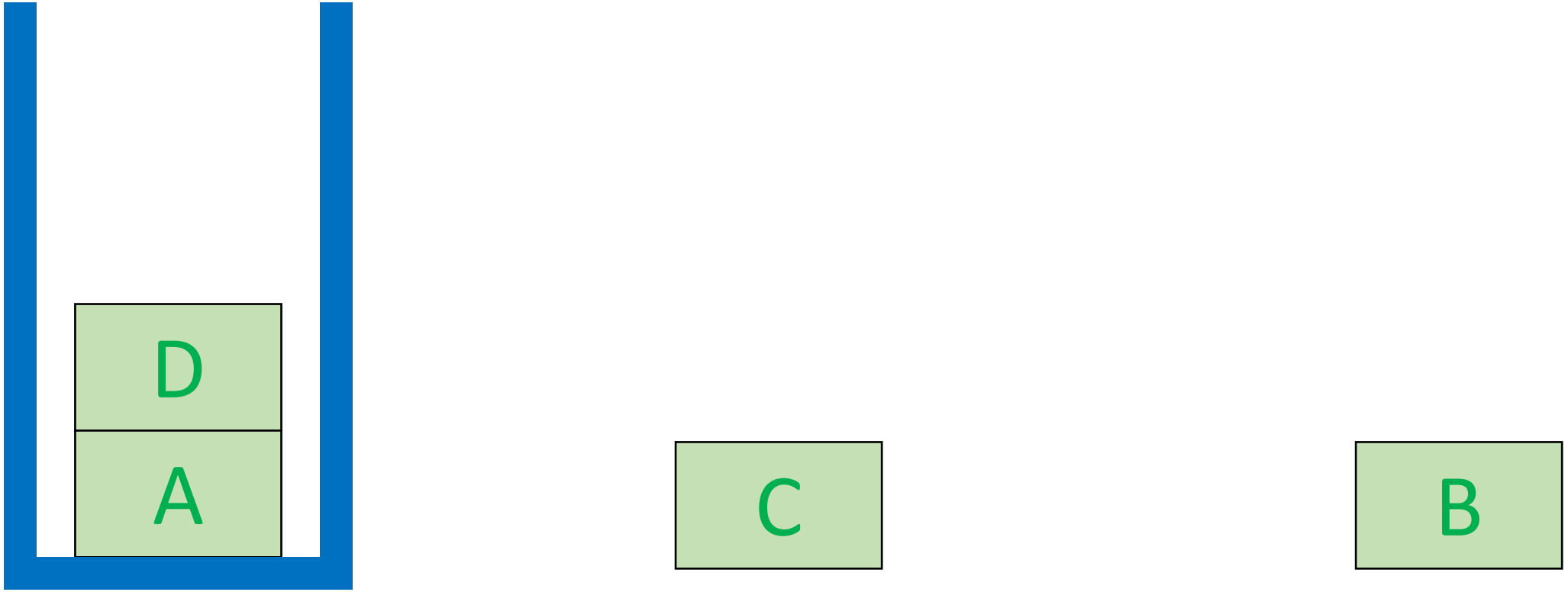
Стек



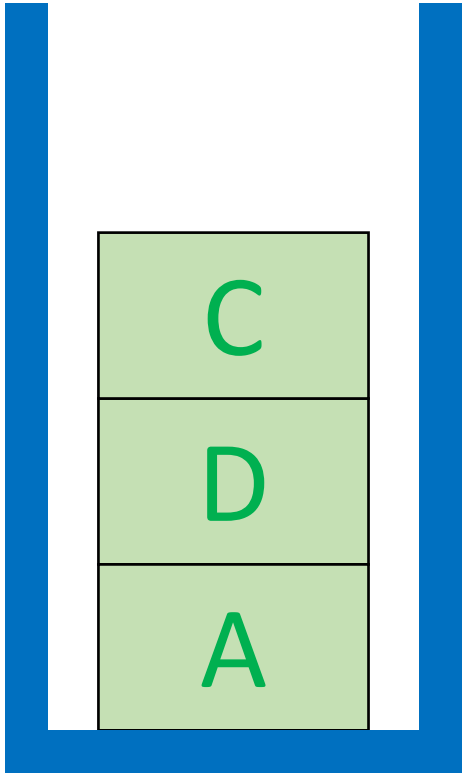
Стек

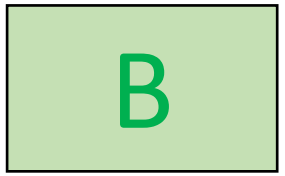


Стек



Стек





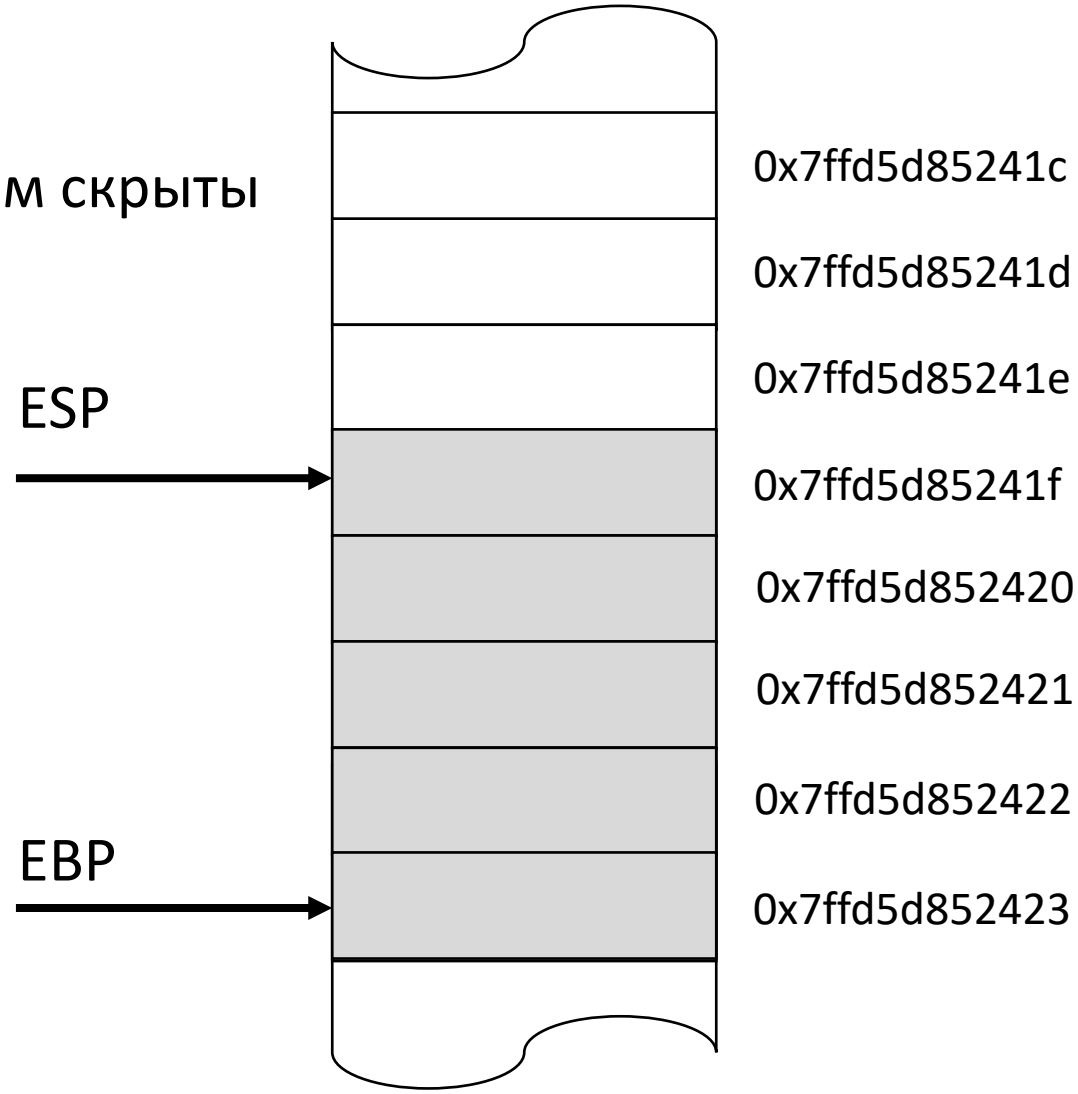
Стек

Для программиста операции работы со стеком скрыты

На языке Ассемблер (x86) используются специальные регистры

ESP – указатель на вершину стека

EBP – указатель на начало стека



Куча

Куча - название структуры данных, с помощью которой реализована динамически распределяемая память приложения.

Выделение памяти

Стек	Куча
Память распределяется по методу (LIFO)	Память распределяется в случайном порядке
Нет необходимости освобождения памяти	Необходимо освобождать память
Размер: **1 МБ** (по умолчанию Windows)	Размер: **4 ГБ и более**

Выделение памяти

`void *malloc(size_t size)`	Размещение блоков памяти
`void *realloc(void *memblock, size_t size);`	Повторное выделение блоков памяти.
`void *calloc(size_t number, size_t size);`	Выделяет массив в памяти и инициализирует его элементы значением 0.
`void free(void *ptr)`	Освобождает блок памяти.

Выделение памяти

Тип `void *`

```
void *malloc(size_t size)
```

Размер выделенной памяти

```
void free(void *ptr)
```

Размера памяти **<u>Нет</u>**

Выделение памяти

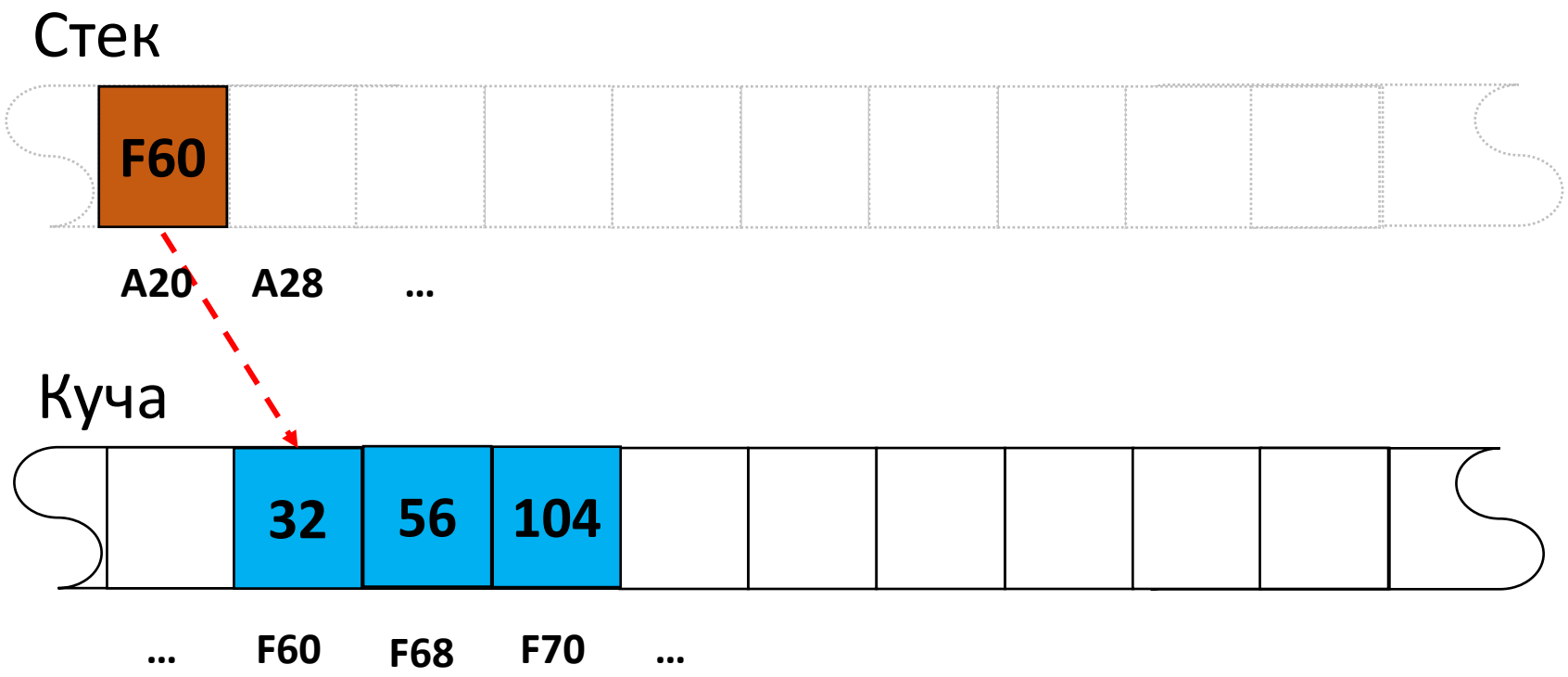
```
int N = 3;
int arrMax = 100, arrMin = 0;

double* arr = (double*)malloc(N * sizeof(double));

for(int i = 0; i < N; ++i)
        arr[i] = arrMin + (arrMax - arrMin) * ((double)rand() / RAND_MAX);

printf("&arr = %p\n", &arr);

for (int i = 0; i < N; ++i)
        printf("&arr[%d] = %p\n",i, &arr[i]);
for (int i = 0; i < N; ++i)
        printf("%lf\n", arr[i]);

free(arr);
```

```
&arr =          00000037405CFA20
&arr[0] =       0000023344359F60
&arr[1] =       0000023344359F68
&arr[2] =       0000023344359F70
0.125126
56.358531
19.330424
```

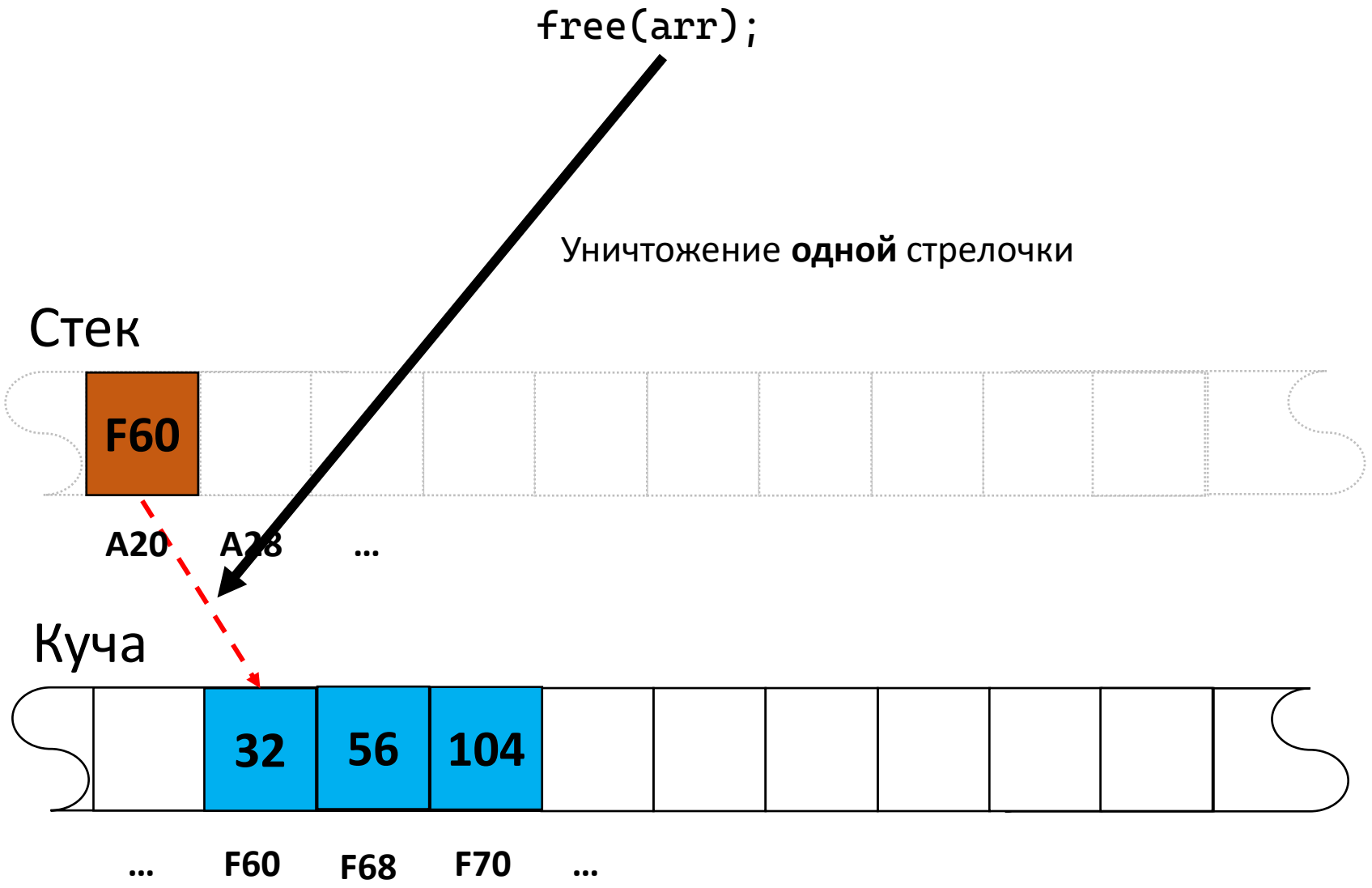
Выделение памяти

```
double* arr = (double*)malloc(N * sizeof(double));
```

```
&arr =          00000037405CFA20
&arr[0] =       0000023344359F60
&arr[1] =       0000023344359F68
&arr[2] =       0000023344359F70
```



Выделение памяти



Пример

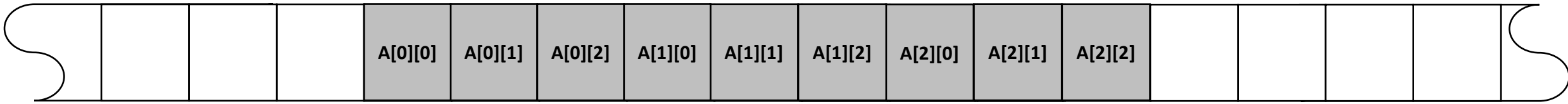
```
#include <stdio.h>
#include <stdlib.h>

const int N = 10;

void input_array(double *x)
{
     int arrMax = 100, arrMin = 0;
     for (int i = 0; i < N; i++)
          x[i] = arrMin + (arrMax - arrMin) * ((double) rand() / RAND_MAX);
}

void print_array(double* x)
{
     for (int i = 0; i < N; i++)
          printf("%lf ",x[i]);
     printf("\n ");
}

int main()
{
     double* array_heap = (double*)malloc(N * sizeof(double));
     input_array(array_heap);
     print_array(array_heap);
     free(array_heap);
     return 0;
}
```

Теперь можно!

```
#include <stdio.h>
#include <stdlib.h>

void input_array(double *x, int N)
{
     int arrMax = 100, arrMin = 0;
     for (int i = 0; i < N; i++)
     x[i] = arrMin + (arrMax - arrMin) * ((double) rand() / RAND_MAX);
}

void print_array(double* x, int N)
{
     for (int i = 0; i < N; i++)
          printf("%lf ",x[i]);
     printf("\n ");
}

int main()
{
     int arr_len;
     scanf_s("%d", &arr_len);          //размер массива указываем уже после запуска программы
     double* array_heap = (double*)malloc(arr_len * sizeof(double));
     input_array(array_heap, arr_len);
     print_array(array_heap, arr_len);
     free(array_heap);
     return 0;
}
```

Работа с двумерными массивами

A[0][0]	A[0][1]	A[0][2]
A[1][0]	A[1][1]	A[1][2]
A[2][0]	A[2][1]	A[2][2]



Работа с двумерными массивами

```
#include <stdio.h>
#include <stdlib.h>

void input_array(double *x, int X_arr, int Y_arr)
{
     int arrMax = 100, arrMin = 0;
     for (int i = 0; i < X_arr; i++)
          for (int j = 0; j < Y_arr; j++)
                    x[j + i * X_arr] = arrMin + (arrMax - arrMin) * ((double) rand() / RAND_MAX);
}

void print_array(double* x, int X_arr, int Y_arr)
{
     for (int i = 0; i < X_arr; i++) {
          for (int j = 0; j < Y_arr; j++)
                    printf("%lf ", x[j + i * X_arr]);
          printf("\n ");
     }
}

int main()
{
     int arr_X, arr_Y;
     scanf_s("%d%d", &arr_X, &arr_Y);
     double* array_heap = (double*)malloc(arr_X * arr_Y * sizeof(double));
     input_array(array_heap, arr_X, arr_Y);
     print_array(array_heap, arr_X, arr_Y);
     free(array_heap);
     return 0;
}
```

Работаем с двумерным массивом как с одномерным

Работа с двумерными массивами

int **X → Указатель на указатель

```
int A = 10;
int* p1 = &A;
int** p2 = &p1;
printf("A = \t%d\n", A);
printf("p1 = \t%p\n", p1);
printf("p2 = \t%p\n", p2);
printf("*p2 = \t%p\n", *p2);
printf("*p1 = \t%d\n", *p1);
printf("**p2 = \t%d\n", **p2);
```

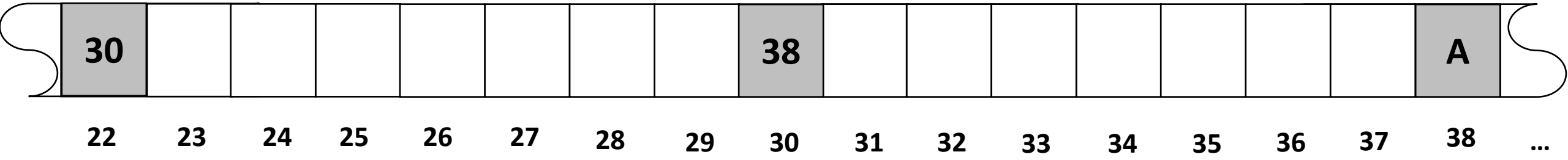
```
A =     10
p1 =    0000002457CFFD38
p2 =    0000002457CFFD30
*p2 =   0000002457CFFD38
*p1 =   10
**p2 =  10
```

Работа с двумерными массивами

int **X → Указатель на указатель

```
int A = 10;
int* p1 = &A;
int** p2 = &p1;
printf("A = \t%d\n", A);
printf("p1 = \t%p\n", p1);
printf("p2 = \t%p\n", p2);
printf("*p2 = \t%p\n", *p2);
printf("*p1 = \t%d\n", *p1);
printf("**p2 = \t%d\n", **p2);
```

```
A =     10
p1 =    0000002457CFFD38
p2 =    0000002457CFFD30
*p2 =   0000002457CFFD38
*p1 =   10
**p2 =  10
```

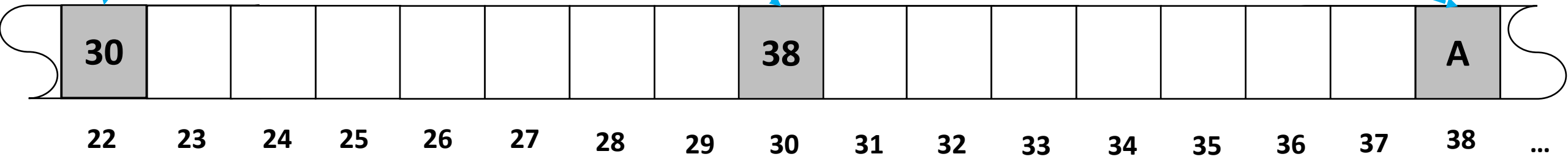


Работа с двумерными массивами

int **X → Указатель на указатель

```
int A = 10;
int* p1 = &A;
int** p2 = &p1;
printf("A = \t%d\n", A);
printf("p1 = \t%p\n", p1);
printf("p2 = \t%p\n", p2);
printf("*p2 = \t%p\n", *p2);
printf("*p1 = \t%d\n", *p1);
printf("**p2 = \t%d\n", **p2);
```

```
A =     10
p1 =    0000002457CFFD38
p2 =    0000002457CFFD30
*p2 =   0000002457CFFD38
*p1 =   10
**p2 =  10
```



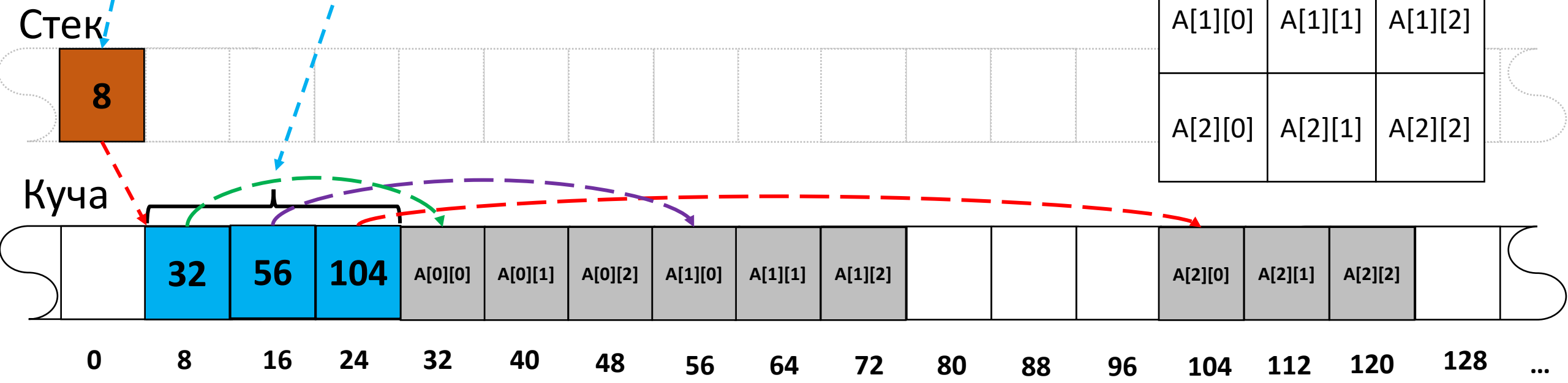
Выделение памяти под двумерный массив

Число строк

```
double** array_heap = (double**)malloc(arr_X * sizeof(double*));
for (int i = 0; i < arr_X; ++i)
{
        array_heap[i] = (double*)malloc(arr_Y * sizeof(double));
}
```

Число столбцов

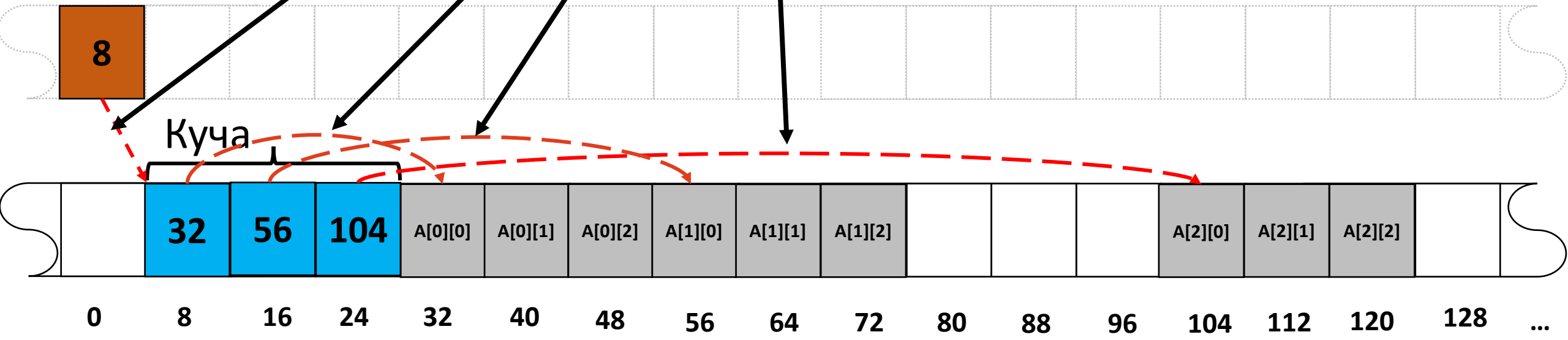
A[0][0]	A[0][1]	A[0][2]
A[1][0]	A[1][1]	A[1][2]
A[2][0]	A[2][1]	A[2][2]



Выделение памяти под двумерный массив

```
for (int i = 0; i < arr_X; i++)
    free(array_heap[i]);
free(array_heap);
```

Необходимо удалить **все** стрелочки



Работа с двумерными массивами

```
#include <stdio.h>
#include <stdlib.h>
void input_array(double** x, int X_arr, int Y_arr){
     int arrMax = 100, arrMin = 0;
     for (int i = 0; i < X_arr; i++)
          for (int j = 0; j < Y_arr; j++)
                    x[i][j] = arrMin + (arrMax - arrMin) * ((double)rand() / RAND_MAX);
}
void print_array(double** x, int X_arr, int Y_arr){
     for (int i = 0; i < X_arr; i++) {
          for (int j = 0; j < Y_arr; j++)
                    printf("%lf ", x[i][j]);
          printf("\n ");
     }
}
int main(){
     int arr_X, arr_Y;
     scanf_s("%d%d", &arr_X, &arr_Y);
     double** array_heap = (double**)malloc(arr_X * sizeof(double*));
     for (int i = 0; i < arr_X; ++i){
          array_heap[i] = (double*)malloc(arr_Y * sizeof(double));
     }
     input_array(array_heap, arr_X, arr_Y);
     print_array(array_heap, arr_X, arr_Y);
     for (int i = 0; i < arr_X; i++)  // цикл по строкам
          free(array_heap[i]);   // освобождение памяти под строку
     free(array_heap);
     return 0;
}
```

Работаем как с обычным двумерным массивом